

Sensitive Data & Information Disclosure — Interview Prep Notes

1. Logical Description

- App reveals data (PII, secrets, tokens, internal system info, financial data) to unauthorized party — at rest, in transit, or in response/output
 - Two buckets:
 - **Exposure** — unintentional over-sharing (excessive API fields, verbose errors)
 - **Leakage** — via side channel (logs, headers, cache, timing, metadata)
 - Root causes: missing/weak encryption, no output filtering (serialize-all), verbose errors, misconfigured storage/access, secrets in code/VCS
-

2. OWASP Category & CWE

- **OWASP 2021: A02:2021-Cryptographic Failures** (renamed from "Sensitive Data Exposure" A3:2017 → refocused on root cause, not symptom) — call this out explicitly in interview
 - Also touches: **A05:2021-Security Misconfiguration** (verbose errors, debug mode), **A01:2021-Broken Access Control** (over-broad data return)
 - **OWASP API Top 10: API3:2019 — Excessive Data Exposure**
 - CWEs:
 - CWE-200 — Exposure of Sensitive Info to Unauthorized Actor (umbrella)
 - CWE-209 — Error Message w/ Sensitive Info
 - CWE-311 — Missing Encryption of Sensitive Data
 - CWE-319 — Cleartext Transmission
 - CWE-312 — Cleartext Storage
 - CWE-532 — Sensitive Info in Log File
 - CWE-538 — File/Directory Info Exposure
 - CWE-598 — Sensitive Data in GET Query String
 - CWE-921 — Storage w/o Access Control
-

3. Common Variants

- Verbose error/stack trace (debug mode on)
- Directory listing enabled

- Sensitive data in URL/GET params (→ Referer/log leakage)
 - Hardcoded secrets in client JS / source maps
 - Excessive API response data (API3)
 - Sensitive data in headers/cookies (no Secure/HttpOnly)
 - Sensitive data logged (PII/secrets in app logs)
 - Source/backup file exposure (`.git` , `.env` , `.bak` , `~` , `.swp`)
 - Cleartext storage (DB, config, backups)
 - Cleartext transmission (HTTP, weak TLS)
 - Autocomplete on sensitive fields
 - Cache leakage (missing `Cache-Control` on sensitive pages)
 - File metadata leakage (EXIF, doc author/path)
 - Version/tech-stack disclosure (`Server` , `X-Powered-By`)
 - Cloud storage misconfig (public S3/Blob/GCS)
 - Third-party script exfiltration
-

4. Attack Surface / Entry Points

- API/HTTP response bodies (JSON/XML — extra fields)
 - Response headers (`Server` , `X-Powered-By` , `Set-Cookie`)
 - Error pages, stack traces, debug endpoints
 - URL/query strings, Referer header
 - Client-side source (JS bundles, source maps `.map`), mobile binaries (strings/decompile)
 - `robots.txt` , `sitemap.xml` , `.well-known/`
 - Directory listings, backup/VCS files (`.git` , `.svn` , `.env` , `.DS_Store`)
 - Application/access/error logs, 3rd-party log aggregators (Sentry, Loggly)
 - Browser/proxy/CDN cache
 - HTML/JS comments
 - Autocomplete-enabled forms
 - Uploaded/served file metadata
 - Cloud storage buckets
 - Nested API objects (password hash, internal IDs, PII beyond need)
 - Analytics/3rd-party JS
-

5. Testing Workflow (Phase-Based, Unified Burp/ZAP/Nuclei)

Phase 1 — Attack Surface Mapping

- **Verify:** enumerate endpoints/files/params that could leak data
- **Manual:** crawl app, check `robots.txt` /sitemap, grep JS for endpoints
- **Burp:** Spider/Crawl → Site map; Content Discovery
- **ZAP:** Spider + AJAX Spider
- **Nuclei:** `-t exposures/` templates
- **Payloads:** `/.git/config`, `/.env`, `/backup.zip`, `/admin/config.php`, `/.DS_Store`

Phase 2 — Response / Header Analysis

- **Verify:** excessive fields in API response vs UI-rendered data; missing security headers
- **Manual:** diff full JSON/XML response against what's rendered
- **Burp:** Repeater (raw response inspection), Logger++
- **ZAP:** Manual Request Editor / Response tab
- **Nuclei:** `misconfiguration/`, `exposed-panels/` templates
- **Payloads:** n/a — look for `password_hash`, `ssn`, `internal_id`, `is_admin`, `role`

Phase 3 — Verbose Error / Debug Disclosure

- **Verify:** stack traces, DB errors reveal paths/versions/queries
- **Manual:** malformed input to trigger 500s, boundary/type fuzzing
- **Burp:** Intruder (error-based payload set), Turbo Intruder
- **ZAP:** Fuzzer
- **Nuclei:** `exposures/` error-disclosure templates
- **Payloads:** `'` (SQLi meta-char), wrong type (string→int), null byte, oversized input

Phase 4 — Client-Side Source Review

- **Verify:** hardcoded secrets, exposed source maps
- **Manual:** view-source, download JS bundles, regex grep for secrets
- **Burp:** Proxy history export + search / grep extension, DOM Invader
- **ZAP:** Passive scan — Information Disclosure rules
- **Nuclei:** `exposures/tokens/`, `exposed-panels/js`
- **Payloads (regex):** `AKIA[0-9A-Z]{16}`, `api[_-]?key`, `Bearer`, `-----BEGIN PRIVATE KEY-----`
`--`

Phase 5 — Storage / Transmission

- **Verify:** cleartext transport, weak TLS config, unencrypted sensitive DB fields

- **Manual:** protocol check (HTTP vs HTTPS), `testssl.sh` / `sslyze`, HSTS check
- **Burp:** mixed-content flags in proxy history
- **ZAP:** passive scan info-leak rules (TLS scan limited)
- **Nuclei:** `ssl/` templates
- **Payloads:** n/a — config/tool-driven

Phase 6 — Metadata / File Exposure

- **Verify:** EXIF/doc metadata, leftover backups, `.git` exposure
- **Manual:** `exiftool`, `git-dumper` on exposed `.git/HEAD`, check `.bak/.old`
- **Burp:** Content Discovery / Intruder w/ backup extension wordlist
- **ZAP:** Forced Browse add-on
- **Nuclei:** `exposures/backups/`, `exposures/configs/`
- **Payloads:** `file.php.bak`, `file~`, `.git/HEAD`, `Thumbs.db`

Phase 7 — Cache Testing

- **Verify:** sensitive/authenticated pages cached by browser/proxy/CDN
 - **Manual:** check `Cache-Control` / `Pragma` on auth'd responses; re-fetch post-logout
 - **Burp:** Response header inspection, Param Miner (cross-check w/ cache poisoning)
 - **ZAP:** Passive scan — cache-control rule
 - **Nuclei:** mostly manual
 - **Payloads:** n/a — verify `Cache-Control: no-store` / `Pragma: no-cache` present
-

6. Evidence to Collect

- Full raw request/response pairs showing exposed data
 - Redacted screenshot/proof (mask real PII — show partial only)
 - Headers showing missing protections (`Cache-Control`, `Set-Cookie` flags)
 - URL/file proving exposed backup/config/source
 - Diff: API response fields vs UI-rendered fields
 - Timestamp, endpoint, method, affected param
 - Data classification impacted (PII/PCI/PHI/credentials)
 - Scale indicator (# records/users affected, if enumerable)
-

7. Good PoC

- Exact repro: endpoint, method, auth level (unauth/low-priv), curl/Burp request

- Redacted response proof (mask real values, e.g., `xxxx-xxxx-xxxx-1234`)
 - Demonstrate scale minimally (2–3 records, not full dump) — ethical/scope-respecting
 - Tie to business impact statement (data type → regulatory framing)
 - No mass exfiltration — proof of concept, not proof of exploitation at scale
-

8. Attack Chains

- Info disclosure → credential stuffing (leaked emails/usernames)
 - Verbose error → SQLi confirmation → full DB dump
 - Source map/JS secrets → API key → cloud account takeover
 - `.git` exposure → source leak → hardcoded creds → lateral movement
 - Excessive API data (API3) → PII harvesting → account takeover (exposed security answers)
 - Internal IP/path disclosure → SSRF target enumeration
 - Session token in URL → Referer leakage → session hijacking
 - Stack trace → framework/version fingerprint → known CVE exploitation
 - Excessive data exposure + BOLA/IDOR → mass PII breach (compounding effect)
-

9. Common False Positives

- Generic/public info in error (non-sensitive)
 - Version disclosure with no known exploitable CVE (informational only)
 - Data already intentionally public via legit feature
 - Masked/truncated data misread as full exposure (e.g., last 4 card digits — PCI compliant)
 - Sensitive-looking field name but test/dummy data
 - Cached static, non-sensitive assets
-

10. Prerequisites for Exploitability

- Attacker can reach endpoint/file (network/auth boundary crossed)
 - Returned data is genuinely sensitive/non-public
 - No compensating control blocks access (WAF, gateway filter)
 - Data is usable for further attack (not already public elsewhere)
-

11. Attacker Achievements

- PII harvesting → identity theft, phishing
- Credential/token theft → account takeover
- Source code/secrets → deeper compromise (cloud/DB creds)
- Business intel leakage (internal emails, org structure)
- Regulatory-scope data exposure (financial, health records)
- Recon for further attacks (tech-stack fingerprinting)

12. CIA Impact by Variant

Variant	C	I	A	Example CVSS Vector
Verbose error / version disclosure only	Low	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N (~5.3)
Excessive API data exposure (other users' PII)	High	None	None	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N (~7.5)
Excessive exposure + BOLA (mass PII)	High	Low	None	AV:N/AC:L/PR:L/UI:N/S:C/C:H/I:L/A:N (~9.1)
Source code / secrets leak (<code>.git</code> , hardcoded keys)	High	High	High	AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H (~9.8, if leads to RCE)
Cleartext transmission of credentials	High	None	None	AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N (~7.4, MITM req'd)
Cleartext storage (DB)	High	None	None	AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:N/A:N (varies w/ access)
Cache leakage of authenticated data	Medium	None	None	AV:N/AC:L/PR:N/UI:R/S:U/C:L/I:N/A:N (~4.3–6.5)

13. Typical CVSS

- Range: **Medium** → **Critical**, driven entirely by data sensitivity + reachability
- Version/banner disclosure alone → Low/Medium (~4–5)
- Other-users' PII via API → High (~7.5–9.1)
- Secrets/source leak enabling further compromise → Critical (~9.8)
- Always justify: **scope change (S:C)** applies if exposure of one user's data → compromise/impact beyond original security scope (e.g., leads to admin takeover)

14. Business Impact — Financial App

- **PCI-DSS** violation if cardholder data exposed → fines, loss of processing rights
 - **GDPR/DPDP** fines for PII leakage (up to 4% global revenue under GDPR)
 - Regulatory breach notification duty (RBI/SEBI in India, GLBA/state laws in US)
 - Reputational damage, customer churn, trust loss
 - Enables downstream fraud (ATO, unauthorized transactions)
 - Litigation/class-action exposure
 - Audit findings → loss of certifications (ISO 27001, SOC 2, PCI attestation)
-

15. Indicators / Signals

- 500 errors with stack traces in response body
 - Response contains fields not rendered in UI
 - `robots.txt` /sitemap reveals sensitive paths
 - "Index of /" directory listing pages
 - `Server` / `X-Powered-By` reveal exact versions
 - `.git`, `.env`, `.bak` return `200`, not `404`
 - JS bundles contain `API_KEY` / `secret` patterns
 - Missing `Cache-Control` on authenticated endpoints
 - Session tokens/PII visible in URL, Referer, or logs
-

16. Root Cause (Code-Level)

- Serializing full DB entity/ORM object directly to response — no DTO/allow-list
- Debug/dev mode enabled in production (verbose stack traces)
- Missing try/catch → unhandled exceptions bubble to client
- Logging full request/response objects, including passwords/PII
- Hardcoded secrets in source, committed to VCS
- No encryption at rest for sensitive DB columns
- HTTP instead of HTTPS; missing HSTS
- Default server config (dir listing on, debug console exposed)

```
// Anti-pattern
function getUser(id):
    user = db.query("SELECT * FROM users WHERE id=?", id)
    return jsonResponse(user)    // leaks password_hash, ssn, is_admin, internal_notes
```

17. Remediation / Secure Coding

- **DTO/response schema** — explicit field allow-list, never serialize entity directly
- Disable debug mode in prod; generic error + correlation ID; detailed log server-side only
- Encrypt at rest (AES-256) + in transit (TLS 1.2+, HSTS)
- Centralized logging w/ PII redaction/masking (log scrubbers)
- Secrets management (Vault/KMS) — no hardcoding; pre-commit hooks (`gitLeaks` , `git-secrets`)
- Least privilege / need-to-know on returned data
- `Cache-Control: no-store` on sensitive responses
- Strip default/identifying headers (`Server` , `X-Powered-By`)
- Exclude backup/VCS artifacts from deployment (build pipeline hygiene)

```
// Pattern: explicit allow-list DTO
function getUser(id):
    user = db.query(...)
    return jsonResponse({ id: user.id, name: user.name, email: user.email })
```

Quick per-language field-filtering references:

- **Java/Spring:** `@JsonIgnore` , `@JsonView` , dedicated DTO class
- **Node.js:** manual field mapping / `class-transformer` `@Exclude()`
- **Python (DRF):** `serializers.ModelSerializer` w/ explicit `fields = [...]`
- **C#/.NET:** `[JsonIgnore]` , AutoMapper w/ explicit `ProjectTo<DTO>()`

18. Compensating Controls (if root cause fix delayed)

- WAF/API gateway response filtering (regex/allow-list at edge)
- Rate limiting on endpoint (reduce mass extraction)
- Enhanced monitoring/alerting (DLP on response patterns)
- Temp access restriction (auth requirement, IP allow-list)
- Disable verbose errors at web-server/framework config level (even if app-level fix pending)
- CDN/WAF block known sensitive paths (`.git` , `.env`)

19. If Recurring — What It Signals

- No secure SDLC gates (SAST/secrets scanning absent from CI/CD)

- No standardized DTO/output-filtering pattern across teams
 - No centralized logging/error-handling framework
 - Code review not scoped for data-exposure checks
 - No data classification policy
 - Reactive vs proactive security culture — same systemic/compounding-risk framing as recurring IDOR (regulatory + exec reporting angle)
-

20. Upstream Improvements

- **SAST:** Semgrep taint rules (PII → log/response sink), CodeQL sensitive-dataflow queries
 - **Secrets scanning:** `gitleaks` / `truffleHog` — pre-commit + CI gate
 - **Linters:** ban `SELECT *`, enforce DTO usage
 - **Framework protections:** Jackson `@JsonIgnore`, .NET `[JsonIgnore]`, serializer allow-lists
 - **Centralized error-handling middleware** (generic client-facing errors)
 - **Security headers middleware** globally applied (`Cache-Control`, HSTS)
 - **Design checklist:** data classification review at API-contract/design stage
 - **CI schema-diff tests** — contract test flags new/unexpected response fields
-

21. Sample Triage Response

Thanks for the report. Confirmed: `GET /api/v1/users/{id}` returns `ssn`, `password_hash`, and `internal_notes` fields not used by the client UI — classified as Excessive Data Exposure (API3:2019 / CWE-213/CWE-200). Severity assessed as **High (CVSS 7.5)** given unauthenticated-reachable PII. Escalating to engineering for DTO-based response filtering. Will update once fix is deployed to staging for retest.

22. Sample Developer Remediation Note

Issue: `/api/v1/users/{id}` serializes full `User` entity → exposes `password_hash`, `ssn`, `internal_notes` to client. **Fix:** Replace entity serialization with `UserResponseDTO` containing only `id`, `name`, `email`. Add contract test asserting response schema. Do not use `to_json()` / generic serializers on entities carrying PII. **Verify:** Retest via Burp Repeater — confirm restricted field set in response; add Semgrep rule to flag direct entity serialization in future PRs.

23. Real-World Examples

- **Twitter (2022):** API endpoint over-returned account data tied to submitted email/phone → *attackers sold data of 5.4 million Twitter users, followed by 400 million users' data scraped and sold on the dark web*; API3-style excessive data exposure via enumeration
 - **CVE-2020-14179 (Jira):** unauthenticated info disclosure — flagged in a DoD bug bounty report where *internal fields like Project, Status, Limits, Creator, and dates were exposed via a subdomain vulnerable to this CVE*
 - **Uber (2016):** hardcoded AWS credentials committed to a private GitHub repo → breach of 57M user/driver records (classic secrets-in-source root cause)
 - **Coinbase bug bounty:** *a critical API flaw earned a \$250,000 bug bounty payout, with Coinbase resolving it within 6 hours of notification* — shows severity/payout scale for API-level data-exposure-class bugs in fintech
 - **Panera Bread (2018):** unauthenticated API returned full customer records (name, email, last 4 of card) — classic excessive data exposure, publicly disclosed after vendor delay
-

30-Second Interview Explanation

"Sensitive Data & Information Disclosure is when an app reveals data it shouldn't — either by design flaw (API returns more fields than the UI uses) or by leakage (verbose errors, exposed `.git` / `.env`, missing encryption). OWASP 2021 reframed this as Cryptographic Failures — A02 — because the fix is root-cause: encrypt properly, filter output via DTOs, disable debug in prod, and never hardcode secrets. Severity depends entirely on data sensitivity — a version banner is Low, other users' PII via an API is High, and leaked secrets that enable further compromise are Critical."